



A Community Shared Workspace
Where great ideas grow!

A Community Credit Exchange System

White Paper

Written by Darren Knipe

A capacity-backed, reserve-anchored, distributed storage and services
economy for CIC networks

Document Classification

Version: 1.0

Produced: 01/12/2025

Updated: 04/12/2025

Document Type: Technical Financial White Paper

Standards Orientation: ISO/IEC 27001, ISO/IEC 22301, ISO 9001, HMRC Barter Rules (UK), CIC
Regulator Framework

Prepared For: Community Interest Companies (CICs), socio-enterprise, business, and social
investors

Security Classification: Public

Status: Draft for Review



A Community Shared Workspace
Where great ideas grow!

Executive Summary

The **Community Credit Exchange System** is a capacity-backed community economic protocol originally designed for a federated inQbeta CIC framework.

It has since become clear that any community organisation with time- or volume-based capacity resources can implement this model.

Therefore, the system is being made available as an **open-source** standard.

Why the Need

Conventional economics describes wealth using the relationship between money supply and velocity:

$$Wealth = MoneySupply \times Velocity$$

Where:

MoneySupply = total amount of money available to spend in a community

Velocity = how many times that money is used (exchanged) in a given period

Simple Example

If I have **£10**, and:

- I pay a hairdresser
- the hairdresser buys vegetables
- the greengrocer buys fuel

Then:

$$Wealth = \text{£10} \times 3 = \text{£30}$$

By moving one **£10** note three times, we created **£30** of community economic activity.



A Community Shared Workspace
Where great ideas grow!

Replace Money With Capacity Resources

If we replace money with resource units (hours, storage, sessions), then the wealth of our community increases when we circulate the services we already have.

This model overcomes limitations found in earlier exchange systems such as LETS because it provides a self-balancing and auditable approach to the **terms of trade**.

What the System Enables

At its core, the system allows a community organisation to:

- Mint and circulate digital credits backed by real, deliverable service capacity — never by financial speculation
- Track and audit all credit circulation using double-entry accounting and hash-linked transaction history
- Underwrite credits with cash reserves in CIC bank accounts to meet cost-of-service obligations and prevent over-issuance
- Distribute capacity dynamically using replication rules across hosting nodes to increase resilience
- Use grants as reserve multipliers to increase capacity access without debt or speculative risk

Credits Represent Claims on Real Services

Including (but not limited to):

- Workspace and hot-desking
- Training and educational sessions
- Wellbeing, therapy, and social support appointments
- Digital storage and distributed hosting
- Studio space, tools, or workshop rental



A Community Shared Workspace
Where great ideas grow!

System Design Principles

1. **Non-speculative**
credits do not appreciate or pay interest
2. **Reserve-backed**
issuance tied to **Cost-Of-Service** reserves
3. **Capacity-limited**
governed by measurable supply, not theoretical demand
4. **Audit-traceable**
full ledger visibility and accountability
5. **Compliant**
UK CIC rules and HMRC barter principles

The architecture aligns with international standards:

- ISO/IEC 27001 — information security and access control
- ISO/IEC 22301 — business continuity and resilience
- ISO 9001 — quality and process governance



A Community Shared Workspace
Where great ideas grow!

Critical Evaluation (Peer Review Perspective)

While promising, several areas require stress testing before deployment in diverse community settings.

Market Volatility

The model assumes predictable usage rates inside a fixed time window, but real-world demand is variable.

Formulas must be run under:

- bull conditions (high utilisation)
- stable conditions
- bear conditions (declining utilisation, zero-demand)

Reserve Adequacy & Liquidity

Cost-Of-Service is not static. A robust model should evaluate:

- reserve depletion
- sudden service demand spikes
- periods with no new income
- grant exhaustion

This may justify adaptive reserve ratios.

Behavioural Dynamics & Hoarding Prevention

Credits do not expire, but capacity does (especially time-based services).

Dynamic issuance rules based on unbooked credits are recommended and require empirical validation.



A Community Shared Workspace
Where great ideas grow!

Network Liquidity

Assuming inter-CIC capacity availability needs network simulation, including:

- scarcity
- outage
- hardware failure
- uneven distribution of demand

Governance in Open-Source Systems

A reviewer will ask:

- Who controls formula updates?
- Who approves changes to reserve ratios?
- What is the governance mechanism for forks?

Mathematics alone cannot enforce decision rules.

Summary of Value Proposition

The **Community Credit Exchange System** offers a non-speculative, capacity-limited, and auditor-friendly alternative to mutual credit schemes and local currencies. It treats service capacity as collateral, combining:

- capacity-based issuance
- reserve discipline
- distributed storage resilience
- transparent governance rules

If properly stress-tested across different demand conditions and supported by accountable governance, the model presents a scalable method of community wealth creation without debt, profit extraction, or speculative instruments.



A Community Shared Workspace
Where great ideas grow!

Design Principles

The **Community Credit Exchange System** is governed by eight design principles. Each principle defines a constraint to prevent misuse and ensure stability under real-world conditions.

1. Capacity Integrity

Meaning:

Credits must always correspond to deliverable service units — hours, sessions, storage, or equipment use.

Implication:

If a capacity unit cannot be delivered within an agreed time window, no credit can be minted for it.

2. Financial Solvency

Meaning:

Credits must be underwritten by cash reserves sufficient to cover **Cost-of-Service (COS)**.

Implication:

If reserves fall below required levels, issuance automatically stops until balance is restored.

3. Regulatory Transparency

Meaning:

All economic actions must be logged, auditable, and reconciled using double-entry accounting and hash-linked records.

Implication:

Every credit movement leaves a permanent trace, preventing hidden transfers, unrecorded issuance, or ledger manipulation.

4. Distributed Resilience

Meaning:

Digital data must exist in multiple physical locations to prevent loss or localised failure.

Implication:

Storage is automatically replicated to three different CIC nodes, and no two replicas may reside on the same physical host.



A Community Shared Workspace
Where great ideas grow!

5. Non-Speculative Value

Meaning:

Credits are not investment instruments, do not appreciate, and may not generate yield or interest.

Implication:

The value of a credit is always fixed by capacity and reserves, not market sentiment or trading activity.

6. Velocity Over Hoarding

Meaning:

Community wealth increases when credits circulate, not when they are stored or accumulated.

Implication:

Issuance is governed by unbooked credit thresholds (dynamic issuance and utilisation controls), preventing hoarding and ensuring booking availability.

7. Deterministic Minting & Clearing

Meaning:

All credit creation, revenue allocation, and reserve movements must follow fixed mathematical rules rather than human judgement or discretionary decision-making.

The system determines when minting is allowed, how money is split at transaction time, and how reserves are protected using predefined formulas.

Implication:

Credit issuance stops automatically when any safety constraint is violated, revenue is divided the same way every time money enters, and reserve balances are adjusted through deterministic clearing logic.

No board vote, staff intervention, or policy interpretation is required for minting, reserve allocation, or ledger settlement — all actions are computed at the point of sale and enforced consistently across the entire network.

$$GP = COS + DIV + (GP \times PR) + Local$$

And

$$C_{mintable_{now}} = \min(IR \times K_{window}, MaxCreditFraction \times K_{window}, R/COS)$$

No governance vote or manual override is required.



A Community Shared Workspace
Where great ideas grow!

8. Principle: Separation of Credit Ownership & Booking Access

Meaning:

Credits enable participation in the system but do not guarantee immediate capacity availability.

Implication:

If capacity is saturated, members may book later or choose another CIC. Financial solvency is preserved regardless of booking pressure.

9. Data Sovereignty & Consent

Meaning:

Storage, encryption, and replication must comply with UK GDPR and retain member control over personal data.

Implication:

Data must be encrypted at rest, encrypted in transit, and removable on request without breaking hash linkage or reserve records.

10. ISO-Style Control Framework

Meaning:

The system adopts formal control thinking rather than ad-hoc local policies.

Implication:

Design and deployment must align with established standards, including:

- ISO/IEC 27001 — information security controls
- ISO/IEC 22301 — service continuity and disaster recovery
- ISO 9001 — quality management and procedural governance

Every action must be measurable, reviewable, and repeatable.

11. Separation of Credit Ownership & Booking Access

Meaning:



A Community Shared Workspace
Where great ideas grow!

Owning credits gives a member the right to participate in the system and request bookings, but it does not guarantee that a service will be available at a specific time or location. Credits represent the ability to access capacity when it becomes available, not a promise of immediate delivery.

Implication:

If all available capacity is already booked, the member may queue, choose a different time, or select a different CIC. Financial stability is protected because credits do not compel any CIC to deliver a service at a guaranteed moment. Availability depends on real capacity, not on the number of credits in circulation.

Legal Classification: Credits as Eligibility Tokens

Credits issued under the Community Credit Exchange System provide eligibility for service, not guaranteed performance at a specific time. Ownership of credits allows booking attempts within published booking windows but does not create a legal obligation for immediate access.

When capacity is unavailable, the member can:

- book later,
- wait for the next opening,
- or choose another CIC venue.

The absence of immediate availability does not count as a failure to provide service. Credits do not constitute debt, financial securities, vouchers, gift cards, or prepaid instruments under UK law, because they are not a guaranteed entitlement to redeem a service at a fixed date or time.

This keeps the system outside regulation for:

- financial services,
- payment services,
- e-money,
- and investment instruments.

Credits are considered **non-redeemable participation units** within a community trading system backed by capacity and reserves.



A Community Shared Workspace
Where great ideas grow!

Key Legal Points (for CIC documentation)

- Credits do **not** promise delivery at a specific time.
- Credits **do not expire**, but booking windows may.
- Booking is always subject to capacity.
- Lack of capacity is not a breach of contract.
- No stored value liability is created.

By framing credits this way, the system avoids the main triggers for financial regulation in the UK.

Neuro-Friendly Explanation

Credits are like money in a café:

- having cash doesn't guarantee someone will make you a coffee right now if the café is closed or full,
- but you can come back later or go to another café.

This keeps the system easy to understand and fair without risking financial or legal problems.

If you want, I can:

- integrate this directly into your existing Design Principles section,
- format it as a **box sidebar** for visual clarity,
- or generate a **Terms of Service** version suitable for public user agreement on your Next.js frontend.



A Community Shared Workspace
Where great ideas grow!

Additional Notes

These principles:

- are non-negotiable constraints, not optional features
- represent boundary conditions in both software design and community governance
- allow the system to be stress-tested mathematically and behaviourally

A peer reviewer would recognise these as normative assertions that need empirical validation in:

- bull utilisation scenarios
- neutral demand periods
- bear / low utilisation conditions
- grant depletion environments
- failure cascades (e.g. storage node offline)

Fix: Replace “Slack rules” with “dynamic issuance and utilisation controls.”



A Community Shared Workspace
Where great ideas grow!

Core Definitions

Credit

Meaning:

A digitally recorded right to receive one unit of a defined service.

Implication:

A credit is not money. It is a claim on future capacity that can be booked, transferred, or traded.

Capacity Token

Meaning:

A representation of one deliverable service unit (e.g. 1 hour of desk use, 1 GB of storage, 1 wellbeing session).

Implication:

- Tokens represent what exists
- Credits represent who can claim it
- A token must be deliverable within a defined time window

Service Plugin

Meaning:

A configuration that defines:

- service unit type
- capacity limits
- time windows
- cost-of-service model
- reserve ratio
- routing rules
- slack settings for issuance control

Implication:

Plugins convert real-world services into mathematical issuance rules that can be implemented consistently across any platform.



A Community Shared Workspace
Where great ideas grow!

Reserve Ratio

Meaning:

The percentage of cash reserve required per credit issued, based on the cost of delivering that service (**Cost-of-Service / COS**).

Implication:

If reserves drop below required levels, new credits cannot be issued. This prevents insolvency and over-promising.

Replica

Meaning:

A stored digital object copied to a distinct physical node controlled by a separate organisation within the network.

Implication:

Three replicas guarantee data resilience, survivability, and continuity, even if one or two nodes fail.

Network Capacity Market

Meaning:

The system's ability to discover, route, and allocate unused capacity (time or storage) across multiple participating CICs.

Implication:

If one local CIC runs out of capacity, the system can automatically assign replicas or bookings to another CIC with available resources.

Unbooked Credits

Credits held by members that are not yet attached to a specific booking.
This is the actor in anti-hoarding control.

Slack

A percentage of capacity deliberately left unallocated to ensure fair last-minute access within the booking window.

Booking Window

The time period in which a capacity token must be deliverable (default 7 days).
Tokens outside the window must not be issued as credits.



A Community Shared Workspace
Where great ideas grow!

System Architecture

The system consists of six major layers:

1. **Local CIC Nodes**
Deliver services, mint credits within capacity rules, hold reserves
2. **Parent CIC Governance Layer**
Defines global policies, reserve ratios, issuance rules
3. **Credit Exchange**
Implements double-entry ledger and hash chain audit integrity
4. **Distributed Storage Federation**
Triple replica storage using available CIC storage nodes
5. **Bank Reconciliation Agent**
Daily OpenBanking transaction matching and provisional credit settling
6. **Member-Facing Applications**
White-label booking portals, wallets, dashboards

Each layer adopts ISO terminology: **control, audit, risk, continuity, verification, record.**



A Community Shared Workspace
Where great ideas grow!

Service Plugins & Capacity Tokens

A Service Plugin defines how a CIC provides a specific service using digital credits. Each plugin converts real capacity into mathematically bounded issuance, while pricing is governed separately by a Base Rate Policy.

Examples of Service Plugins

- Workspace or hot-desking hours
- Training, meeting, or therapy rooms
- Wellbeing sessions
- Digital storage capacity
- Studio or equipment rental
- Shared tools or fabrication space

Required Plugin Configuration

Each Service Plugin must specify:

- Name
- Unit type (time-based or volume-based)
- Opening schedule
- Capacity per hour or per unit
- Cost-of-Service (COS)
- Reserve ratio
- Revenue share
- Bank account reference
- Credit price policy reference (binds to the Base Rate Policy)

Important:

Pricing is not set in the plugin itself.

Plugins reference the Base Rate Policy, which controls pricing and inflation adjustments.

This ensures consistency across all services.

Base Rate & Inflation Price Policy

Base Rate Definition

The system defines a unit base rate that links currency and credits:

BaseRate_0 = £1 = 1 credit

Where:

- BaseRate is the reference exchange between cash and credits
- Subscript 0 refers to year 0 (starting year)

This base rate is used to price all service units in a consistent way across the CIC.



A Community Shared Workspace
Where great ideas grow!

Inflation Adjustment Rule

To preserve real value over time, the base rate may be adjusted annually in line with inflation or deflation:

$$\text{BaseRate}_{(t+1)} = \text{BaseRate}_t \times (1 + I_t)$$

Where:

- t = current year
- I_t = inflation rate for that year

Example:

If inflation is 5%:

$$\text{BaseRate}_1 = \text{BaseRate}_0 \times 1.05$$

so:

- If BaseRate_0 was £1 per credit
- BaseRate_1 becomes £1.05 per credit

Credits retain purchasing power, without modifying capacity rules or reserve requirements.

Currency Neutrality (Internationalisation)

The Base Rate Policy may include FX rules:

$$\text{BaseRate}_{\text{local}} = \text{FX} \times \text{BaseRate}$$

Example:

- USD Base Rate = $\text{FX } \$\text{GBP} \times \text{BaseRate}$

This preserves:

- one internal credit standard
- multi-currency cash pricing
- no speculative floating of credits

Credits never float.

Only the cash rate changes.

Time-Based Capacity

For services expressed in hours (workspace, training, wellbeing):

Daily Capacity

$$K_d = H \times C_c \times U_t$$

Where:

- H = open hours per day
 - C_c = concurrent capacity
 - U_t = utilisation target (e.g. 0.60)
-

Weekly Capacity Window

$$K_{\text{window}} = K_d \times W$$



A Community Shared Workspace
Where great ideas grow!

Where:

- W = 7 days (default)

Guarantees bookings are deliverable within one week.

Anti-Hoarding Limit

$\text{UnbookedCredits} \leq \text{MaxCreditFraction} \times \text{K_window}$

Example:

- 700 weekly capacity
- 80% unbooked max
- 20% slack guaranteed

This ensures immediate access is always possible.

Storage-Based Capacity

For services expressed in volume:

$S_{\text{free}} = S_{\text{total}} - S_{\text{used}}$

Only free capacity counts.

Storage issuance adapts dynamically:

$C_{\text{mintable}} \leq \text{IR}_{\text{effective}} \times S_{\text{free}}$

Where:

- IR_effective includes stimulus modifiers if applied
-

Cost-of-Service (COS)

$\text{COS} = (\text{FixedCosts} \div \text{Lifespan}) + \text{VariableCosts}$

Practical interpretation:

- Fixed costs amortised
- Running costs included

This is predictable, auditable, and conservative.

Reserve Protection Rule

Before minting credits:

$\text{ReserveBalance} \geq \text{RR} \times \text{COS} \times C_{\text{issued}}$

Where:

- RR = reserve ratio
- COS = cost per unit
- C_issued = circulating credits

If this fails:

- issuance halts



A Community Shared Workspace
Where great ideas grow!

- trigger alerts
 - agent policy actions start
-



Why Pricing Must Be Separate

Separating pricing into a Base Rate & Inflation Policy:

- Prevents pricing drift inside plugins
- Maintains audit traceability
- Simplifies inflation adjustments (one place to change)
- Supports international adoption
- Standardises dividends, grants, FX, and capital multiplier logic

This design is more ISO-compliant and allows CICs to meet HMRC requirements for:

- annual financial review
 - reserve adequacy
 - non-speculative value maintenance
-

Summary

Each service has capacity rules that say how much we can deliver.

Each service has reserve rules that say how much we can promise.

Prices are not set in services but in a separate Base Rate Policy that can change each year with inflation.

Capacity stays stable.

Reserves stay protected.

Pricing stays fair.

Love this question – this is basically the “DNA” of a service plugin 🧬

Below is a clean field list you can use to model a Service Plugin in Dgraph / Modus / any DB.

I’ve grouped fields into:

- Config (what humans design)
- Financial & COS
- Capacity & tokens
- Issuance rules
- Storage specifics (if applicable)
- Live metrics (what agents update)
- Governance & audit

You can treat this as a schema suggestion and trim as needed.

1. Core Identification

- plugin_id – unique ID for the service plugin
 - cic_id – which local CIC this plugin belongs to
 - name – human-readable name (e.g. “Workspace Desks”, “24TB Storage Pool”)
 - description – short description of the service
 - category – enum/string: "workspace" | "training" | "wellbeing" | "storage" | "studio" | "other"
 - service_type – enum: "TIME_BASED" | "STORAGE_BASED" | "ITEM_BASED"
-

2. Unit & Pricing Definition

- unit_label – e.g. "desk-hour", "GB-month", "session"
- unit_quantity – numeric (e.g. 1, 10) – how big each token is
- unit_period – "per_hour" | "per_day" | "per_month" | "per_booking" (for time-based clarity)
- base_rate_currency – e.g. "GBP"
- base_rate_per_credit – base face value (e.g. 1.0 if 1 credit = £1)
- last_base_rate_review_at – timestamp
- inflation_index_source – reference if you later tie to CPI or internal index

(This is where your “£1 = 1 credit, inflation-adjusted” policy lives.)

3. Cost-of-Service (COS) & Financials

- fixed_cost_total – total capital cost allocated to this plugin (£)
- fixed_cost_lifespan_months – months of expected usable life
- variable_cost_per_month – e.g. utilities, cleaning, bandwidth
- cos_per_unit – calculated COS per service unit (one token)
- cos_last_calculated_at – timestamp
- cos_notes – human notes on assumptions (e.g. “electricity estimated at 30p/kWh”)
- reserve_account_id – which bank account tracks reserves for this plugin
- reserve_policy – enum; in your case always "FULL_COS_BACKING"
- reserve_min_buffer – optional policy buffer (e.g. 5% over $\text{COS} \times \text{C_issued}$)

(Agents use these to enforce “ $\text{ReservePerCredit} = \text{COS}$ ” and “ $R \geq \text{COS} \times \text{C_issued}$ ”.)

4. Capacity & Token Space

Time-based

- hours_open_per_day – H
- days_open_per_week – optional (if not 7)
- concurrent_capacity – C_c (desks, seats, rooms, etc.)
- booking_window_days – W (e.g. 7)
- k_d_daily_capacity – derived: $H \times C_c \times U_t$ (but you may store last computed)
- k_window_capacity – derived: K_window (tokens per window)

Storage-based

- storage_total_gb – S_total
- storage_unit_size_gb – e.g. 1 or 10 GB per credit
- storage_tokens_max – derived: $S_{total} \div \text{storage_unit_size_gb}$

(For schema: you can either store derived values or recompute on the fly. I'd at least store storage_unit_size_gb.)

5. Issuance Rules & Anti-Hoarding

- max_credit_fraction – MaxCreditFraction (e.g. 0.8)
- target_utilisation_u_star – U^* (e.g. 0.8)
- alpha_ir_scaling – $\alpha = 1 / U^*$ (can be derived but nice to store)
- ir_min – optional floor (e.g. 0)
- ir_max – usually 1.0
- hoarding_warning_threshold – e.g. 0.6 (60% of MaxCreditFraction)
- hoarding_critical_threshold – e.g. 1.0 (or slightly above)
- stimulus_allowed – bool (can StimulusAgent act on this plugin?)
- stimulus_max_credits_per_period – guardrail for how much stimulus can be applied

(These fields are what IssuanceAgent / StimulusAgent read to make decisions.)

6. Claims & Headroom Fields (Important Fix)

These are the bits that handle the “950 issued vs 1000 capacity” scenario.

- tokens_max_window
 - For time-based: = k_window_capacity
 - For storage-based: = storage_tokens_max
- credits_issued_total – C_issued (live count)
- unbooked_credits – number of credits not yet linked to a booking
- claims_headroom – derived: $\max(0, \text{tokens_max_window} - \text{credits_issued_total})$

(You might store claims_headroom as a computed metric snapshot, but it's conceptually derived.)

7. Live Metrics / Telemetry (Agent-updated)

These fields are updated by MonitorAgent / IssuanceAgent, not by humans:

- current_utilisation_u_t – utilisation ratio (0–1)
- ir_current – current IR value = $\min(\alpha \times U_t, 1)$
- k_d_current – latest daily capacity
- k_window_current – latest weekly capacity

- `max_unbooked_credits` – $\text{MaxCreditFraction} \times K_{\text{window}}$ (for current window)
- `storage_used_gb` – `S_used`
- `storage_free_gb` – `S_free`
- `reserve_balance_cached` – `R` (latest pulled from bank integration)
- `c_reserve_max` – R / COS
- `c_mintable_storage` – $\text{IR} \times \text{storage_tokens_max}$ or $\text{IR} \times \text{S_free}/\text{size}$
- `c_mintable_now` – final safe mintable credits now (the big number on the dashboard)
- `solvency_ok` – bool: $R \geq \text{COS} \times C_{\text{issued}}$
- `hoarding_state` – enum: "OK" | "WATCH" | "OVER"

(These fields are the source of truth for your dashboard.)

8. Stimulus & Behaviour Analytics (Optional but powerful)

- `stimulus_campaign_active` – bool
- `stimulus_campaign_id` – reference to current campaign
- `stimulus_credits_allocated` – number of credits allocated as stimulus
- `stimulus_credits_redeemed` – number actually used
- `average_credit_age_days` – how long credits sit before being used
- `velocity_last_30_days` – number of transfers/bookings per credit in last month

(These help the LLMAdvisorAgent suggest changes and let you test what works.)

9. Governance & Audit

- `is_active` – bool (plugin enabled/disabled)
- `version` – config version (e.g. 1, 2, 3...)
- `created_at` – timestamp
- `created_by_user_id` – who set it up
- `last_reviewed_at` – governance review timestamp
- `last_reviewed_by` – user / board ref
- `iso_scope_tags` – e.g. ["ISO27001", "ISO22301"]
- `risk_rating` – qualitative: "low" | "medium" | "high"
- `notes_governance` – any human notes or board decisions

(AuditAgent can hash-link changes to these fields for immutability.)

10. Optional Dgraph Type Sketch

If you want a rough Dgraph type starting point:

```
type ServicePlugin {
  plugin_id
  cic_id
  name
  description
  category
  service_type

  unit_label
  unit_quantity
  unit_period
  base_rate_currency
  base_rate_per_credit
}
```

last_base_rate_review_at

fixed_cost_total

fixed_cost_lifespan_months

variable_cost_per_month

cos_per_unit

cos_last_calculated_at

reserve_account_id

reserve_policy

reserve_min_buffer

hours_open_per_day

days_open_per_week

concurrent_capacity

booking_window_days

storage_total_gb

storage_unit_size_gb

max_credit_fraction

target_utilisation_u_star

alpha_ir_scaling

ir_min

ir_max

hoarding_warning_threshold

hoarding_critical_threshold

stimulus_allowed

stimulus_max_credits_per_period

tokens_max_window

credits_issued_total

unbooked_credits

current_utilisation_u_t

ir_current

k_d_current

k_window_current

max_unbooked_credits

storage_used_gb

storage_free_gb

reserve_balance_cached

c_reserve_max

c_mintable_storage

c_mintable_now

solvency_ok

hoarding_state

```
is_active
version
created_at
created_by_user_id
last_reviewed_at
last_reviewed_by
iso_scope_tags
risk_rating
notes_governance
}
```

(You'll split these into predicates with types when actually writing the Dgraph schema.)

If you'd like, next I can:

- Turn this into a proper Dgraph schema file (schema.graphql)
- Or map it to a Modus struct in Go for a ServicePlugin agent API.



A Community Shared Workspace
Where great ideas grow!

Absolutely — below is Section 7 rewritten entirely in Google Docs-compatible LaTeX, using the Auto-LaTeX Equations format (\$\$... \$\$) that you confirmed works.

⚠ IMPORTANT

- Every formula is in \$\$... \$\$ block style so Google Docs will convert elegantly.
 - No alignment environments, no curly-escaped math, no $\begin{}$ or arrays — this is tuned specifically for Auto-LaTeX.
 - You can copy-paste the entire section directly into Google Docs.
-

★ CREDIT ISSUANCE RULES (Google Docs LaTeX-Ready)

7. Credit Issuance Rules

Credit issuance is controlled by capacity, reserves, and circulation behaviour.

This ensures credits always correspond to real, deliverable services and that the issuing CIC remains financially solvent at all times.

There are three primary issuance contexts:

- time-based services (rooms, desks, wellbeing, training)
 - storage-based services (SeaweedFS, NAS, cloud buckets)
 - reserve amplification using grants or donated capital
-

7.1 Time Window Enforcement

Weekly capacity window:

\$\$

$$K_{\text{window}} = K_d \times W$$

\$\$

Where:

- $K_d = H \times C_c \times U_t$
- $W = 7$ (default booking horizon in days)

This guarantees every credit corresponds to a bookable service within 7 days.

7.2 Issuance Ratio (IR)

Definition

Dynamic issuance ratio:

\$\$

$$IR = \min(\alpha \times U_t, 1.0)$$

\$\$

Where:

- U_t = rolling utilisation (0–1)
- α = scaling constant chosen by governance



A Community Shared Workspace
Where great ideas grow!

Choosing α

α is derived from a target utilisation U^* :

\$\$

$$\alpha = \frac{1}{U^*}$$

\$\$

Example values:

- $U^* = 0.80 \rightarrow \alpha = 1.25$
- $U^* = 0.70 \rightarrow \alpha = 1.43$
- $U^* = 0.90 \rightarrow \alpha = 1.11$

Neuro-friendly summary

If lots of people are using the service, mint more credits.

If not many are using it, mint fewer.

7.3 Issuance Limit (Multi-Constraint Rule)

Mintable credits are constrained by capacity, hoarding control, and reserves.

Core issuance equation

\$\$

$$C_{\text{mintable_now}} = \min(IR \times K_{\text{window}}; \text{MaxCreditFraction} \times K_{\text{window}}; \frac{R}{\text{COS}})$$

\$\$

Where:

- capacity constraint: $IR \times K_{\text{window}}$
- hoarding constraint: $\text{MaxCreditFraction} \times K_{\text{window}}$
- reserve constraint: $\frac{R}{\text{COS}}$

Constraints defined

Hoarding constraint:

\$\$

$$\text{UnbookedCredits} \leq \text{MaxCreditFraction} \times K_{\text{window}}$$

\$\$

Reserve constraint:

\$\$

$$\text{ReservePerCredit} = \text{COS}$$

\$\$

\$\$

$$\frac{R}{\text{COS}} = C_{\text{reserve_max}}$$

\$\$

Solvency test:

\$\$



A Community Shared Workspace
Where great ideas grow!

$R \geq \text{COS} \times C_{\text{issued}}$
\$\$

Interpretation

The system checks three things before creating credits:

1. Do we have space?
2. Are we leaving room for last-minute bookings?
3. Do we have money to cover delivery?

If all three are “yes,” mint.

If any are “no,” stop.

7.4 Reserve Protection (Full COS Backing)

Cost-of-Service formula:

\$\$

$\text{COS} = \frac{\text{FixedCosts}}{\text{Lifespan}} + \text{VariableCosts}$

\$\$

Reserve per credit:

\$\$

$\text{ReservePerCredit} = \text{COS}$

\$\$

Maximum mintable via reserves:

\$\$

$C_{\text{reserve_max}} = \frac{R}{\text{COS}}$

\$\$

New credits from reserve injection:

\$\$

$\Delta C_{\text{mintable}} = \frac{\Delta R}{\text{COS}}$

\$\$

Neuro-friendly:

Every credit is backed by the full cost to deliver it.

£1 in reserves always creates $1/\text{COS}$ new credits.

7.5 Storage-Based Issuance

Free storage:

\$\$

$S_{\text{free}} = S_{\text{total}} - S_{\text{used}}$

\$\$

Mintable storage credits:

\$\$

$C_{\text{mintable_storage}} = S_{\text{free}} \times \text{IR}$

\$\$



A Community Shared Workspace
Where great ideas grow!

7.6 Grant-Backed Issuance (Reserve Amplification)

Grant multiplier logic:

\$\$

$\Delta C_{\text{mintable}} = \frac{\Delta R}{\text{COS}}$

\$\$

A £1,000 grant @ COS = £2 creates 500 credits.

No speculation. No risk.

7.7 No Credit Expiry

Credits never expire.

Queue safety is handled before issuance, not by penalising users later.

This prevents “use it or lose it” pressure and supports neurodivergent time planning and long-term trust behaviour.

✓ Formatting Notes for Google Docs

- All formulas are in \$\$... \$\$ blocks
 - Auto-LaTeX Equations will convert correctly.
 - No unusual Unicode symbols
 - everything is standard LaTeX.
 - Uses only Greek letters already supported (α).
 - All variables are consistent across Sections 7.1–7.7.
 - You can break up blocks with headings or keep as continuous text.
-

If you’re ready, the next step is:

Option A

“Convert Section 8 to Google Docs LaTeX.”

Option B

“Create LaTeX diagram for issuance logic.”

Option C

“Format whole white paper in Google Docs LaTeX.”

Just tell me which direction you want to go.

```
+-----+
| Utilisation |
|   U_t   |
+-----+
```



A Community Shared Workspace
Where great ideas grow!

$$\begin{array}{c} | \\ | \text{ IR} = \min(\alpha \times U_t, 1.0) \\ \vee \end{array}$$

$$\begin{array}{c} +-----+-----+ \\ | \text{ Issuance Ratio} | \\ | \quad \text{IR} \quad | \end{array}$$

$$\begin{array}{c} +---+-----+---+ \\ | \quad | \\ | \quad | \\ \vee \quad \vee \end{array}$$

$$\begin{array}{c} +---+---+ +---+---+ \\ | \text{ K}_d | \quad | \text{ COS/COS} | \\ +---+---+ +---+---+ \\ | \quad | \\ \vee \quad \vee \end{array}$$

CAPACITY CHECK

$$\begin{array}{c} | \\ \vee \\ +-----+ \\ | \text{ IR} \times \text{K}_{\text{window}} \quad | \\ +-----+ \end{array}$$

HOARDING CHECK

$$\begin{array}{c} | \\ \vee \\ +-----+ \\ | \text{ MaxCreditFraction} \times | \\ | \quad \text{K}_{\text{window}} \quad | \\ +-----+ \end{array}$$

RESERVE CHECK

$$\begin{array}{c} | \\ \vee \\ +-----+ \\ | \text{ R} / \text{COS} \quad | \\ +-----+ \end{array}$$

$$\begin{array}{c} | \\ | \text{ MINIMUM OF THREE} \\ \vee \end{array}$$

+=====+



A Community Shared Workspace
Where great ideas grow!

| C_mintable_now |

+=====+



A Community Shared Workspace
Where great ideas grow!

8. Financial Reserves (COS Protection)

Credit systems collapse when they over-promise.
InQbeta prevents this using Cost-of-Service reserve rules.

Reserve principle:

A CIC may only mint credits when it holds stored GBP sufficient to meet the cost of delivering those services.

8.1 Cost-of-Service Formula

$$COS = \frac{FixedCosts}{Lifespan} + VariableCosts$$

Where:

- FixedCosts = e.g. rent, insurance, maintenance
- Lifespan = annualisation period of service infrastructure
- VariableCosts = costs per unit delivered

This creates a unit cost for each service.

8.2 Reserve Requirement

Credits issued must be backed by a cash reserve at a ratio set in the Service Plugin:

$$ReserveBalance \geq COSReserveRatio \times C_{issued}$$

Where:

- COSReserveRatio (default 0.20 = 20%)

Example:

For £1.00 COS, with 20% reserve, every issued credit requires £0.20 held in bank reserve.
This is non-negotiable.

It is the single most important solvency protection rule.



A Community Shared Workspace
Where great ideas grow!

9. Grant Reserve Amplification

A CIC can receive grant funding, and allocate that to reserves to mint more credits.

Let G = grant allocation to reserve.

Then mintable credits:

$$C_{mintable,grant} = \frac{G}{COSReserveRatio}$$

Example:

$$C_{mintable,grant} = \frac{100}{0.20} = 500$$

This makes each £1 of grant worth 5 service credits, without debt, without interest, and with full audit transparency.

This mechanism is unique and makes grants:

- measurable
- trackable
- levered for community benefit
- impossible to misuse secretly

Because reserve-backed credit tracing is auditable.



A Community Shared Workspace
Where great ideas grow!

10. Banking & Settlement (Instant + ISO Control)

To allow instant member use (ex: printing, booking rooms), credits are issued provisionally upon payment initiation.

Provisional Credits → Confirmed Credits

Workflow:

1. Member initiates Standing Order / BACS / Faster Payment
2. Provisional credits issued immediately
3. Daily settlement agent matches transaction ID
4. Confirmed credits recorded
5. Failed payments → provisional credits destroyed automatically

This workflow allows:

- Instant value to members
- Commission-free operation
- Full reserve security
- ISO 9001 traceability

Banks used:

- Starling Business Account
- Triodos
- Co-operative Bank
- Tide
- Monzo Business

(OpenBanking compatible)

10.1 Reserve Verification

At daily settlement:

$$ReserveBalance_{after} \geq RequiredReserves$$

If not:

- Issuance automatically halts
- Admin alert triggers
- No further credits may be minted until balance restored

This is a hard safety control.



A Community Shared Workspace
Where great ideas grow!

Reserve Consumption Mechanics

(How reserves are reduced when services are actually delivered)

The Community Credit Exchange System treats reserves as a two-way accounting mechanism.

Reserves increase when credits are sold, and reserves decrease when real service work is delivered.

This section defines the formal rules for reserve release, ensuring that protected reserves do not grow indefinitely, and that the system reflects real economic consumption accurately.

11.1 Purpose

Reserves accumulate during sales events to ensure:

- service delivery can be funded,
- dividend obligations are met, and
- system solvency is protected.

However, when real capacity is used (desk hours consumed, storage allocated, equipment rented, etc.), part of the reserve must be released and made available for:

- materials,
- utilities,
- operational costs,
- and associated payables.

Reserve consumption mechanics prevent:

- phantom reserve growth,
 - misleading solvency signals,
 - and artificial inflation of ledger balances.
-

11.2 Two-Event Accounting Model

The system recognises two distinct events:

A. Sale Event

Money arrives → reserves increase.

B. Fulfilment Event

Capacity is consumed → reserves decrease.

Key principle:

Issuance builds liability; fulfilment extinguishes liability.

This mirrors standard financial accounting (revenue recognition and matched cost principles).



A Community Shared Workspace
Where great ideas grow!

11.3 Core Reserve Release Rule

When a service unit is delivered, the system reduces the COS (Cost of Service) reserve by one COS unit:

Reserve release formula:

\$\$

$\text{COS_reserve} := \text{COS_reserve} - \text{COS}$

\$\$

and correspondingly:

\$\$

$\text{COS_spent} := \text{COS_spent} + \text{COS}$

\$\$

This creates a closed loop:

- cosine in
- cosine out
- net effect = reserved for future units only

11.4 Outstanding Reserve Liability

To correctly measure solvency, we track only reserves for unfulfilled credits, not total historical credits sold.

Outstanding liability:

\$\$

$\text{Outstanding_COS} = \text{COS} \times (\text{Credits_sold} - \text{Credits_used})$

\$\$

This ensures reserves represent future obligations only, not historical accumulation.

11.5 Ledger Separation

Three independent tracking ledgers are recommended:

1. COS Reserve Ledger
Records accruals at sale.
2. COS Spend Ledger
Records releases at fulfilment.
3. COS Outstanding Liability Ledger
Tracks total obligations remaining.

These ledgers must not be netted for accurate audit.

11.6 Fulfilment Event Detection

A fulfilment event is triggered by one of the following actions:

- a booking time window is completed,
- storage allocation persists beyond charge interval,
- equipment return is scanned,



A Community Shared Workspace
Where great ideas grow!

- session attended is marked complete.

Events are captured via:

- booking system API,
- SeaweedFS storage allocator timestamps,
- local hardware access systems,
- trainer/therapist attendance logs.

The system must record:

- unit delivered,
 - timestamp,
 - plugin ID,
 - unit price (COS),
 - member identification reference.
-

11.7 Reserve Release Frequency

Systems can release reserves:

- immediately (real-time),
- daily batch,
- weekly clearing window,
- monthly accounting period.

White paper default:

Daily clearing at 02:00

(ISO 22301: avoids peak operational disruption)

11.8 Expenditure Permission Logic

COS release does not directly spend money.

It grants permission to spend money.

Meaning:

- Reserves remain in bank,
- but ledger category changes from “protected” to “spendable”.

This aligns with ISO 9001 “controlled budget” logic.

11.9 Audit Rule (Deterministic)

At any time:

- COS Reserve must equal Outstanding COS Liability

deterministic check:

\$\$

$\text{COS_reserve_balance} = \text{Outstanding_COS}$

\$\$

If not true, the system raises an automatic non-conformance alert.

No human judgement.

No optionality.



A Community Shared Workspace
Where great ideas grow!

11.10 Example – Time-Based Service

Parameters:

- COS per hour: £1.20
- Credits sold: 100
- Credits used: 60

Reserve liability:

\$\$

$\text{Outstanding_COS} = 1.20 \times (100 - 60) = 48$

\$\$

- COS Reserve must be £48
- COS Spend must be £72
- Total sales COS allocation = £120

Reserve is valid and balanced.

11.11 Example – Storage Plugin

Parameters:

- COS per GB-month: £0.50
- Credits sold: 500
- Credits used: 200

Outstanding COS:

\$\$

$\text{Outstanding_COS} = 0.50 \times 300 = 150$

\$\$

At service month-end:

- Reserve decreases by £100
 - £100 moves to COS Spend
-

11.12 Regulatory Rationale

The reserve consumption model:

- prevents artificial inflation of reserve balance,
- supports accurate CIC annual accounts,
- aligns with HMRC “cost matching” principle,
- matches IFRS accrual principles while remaining fully exempt from FCA regulation.

This allows the system to operate as a trading CIC rather than a financial services provider.



A Community Shared Workspace
Where great ideas grow!

11.13 Neuro-Friendly Summary

When someone buys a credit,
we save a bit of money for doing the work later.

When we actually do the work,
we move that saved money into the “spending” pot.

This keeps everything fair and stops the savings pot getting bigger forever.

11.14 Implementation TODOs

- Add FulfilmentAgent to detect service completion events
 - Add ledger entries: COS_spent, COS_outstanding
 - Add dashboard metric: “COS Fulfilment vs COS Liability”
 - Add nightly clearing workflow
 - Add Dgraph predicates for fulfilment events
-

This completes the Reserve Consumption Mechanics section.

If you like, next steps can include:

- Write Section 11.15: Dividend Reserve Consumption
- Produce a flow diagram “Sale → Reserve → Fulfilment → Expenditure”
- Create a simulation sheet tab for reserve consumption over time



A Community Shared Workspace
Where great ideas grow!

Transaction Point Allocation & Revenue Clearing

Deterministic Ledger Model for Credit-Based Community Economics

The Community Credit Exchange System treats the point of sale as the moment where economic reality is written to the ledger.

Credits only become obligations when sold, not when issued.

This section explains the allocation rules, reserve logic, and clearing hierarchy used to protect solvency and ensure transparency.

8.1 Transaction Event

A transaction event occurs when a credit is purchased and payment is received.

At this moment, one gross payment is split into four internal ledger entries:

- Cost-of-Service reserve (COS)
- Capital dividend reserve (DIV)
- Parent CIC revenue (system fee)
- Local CIC revenue (operational surplus)

Credits that are issued but not yet sold do not create reserve requirements.

8.2 Four-Way Revenue Split (Auto-LaTeX Equations)

Let:

- GP = gross price per unit
- COS = cost of service per unit
- DIV = dividend allocation per unit
- PR = parent share rate (e.g. 0.10)

Then:

- COS reserve:
 $COS_reserve = COS$
- Dividend reserve:
 $DIV_reserve = DIV$
- Parent CIC revenue:
 $Parent_share = GP \times PR$
- Local CIC revenue:
 $Local_share = GP - (COS + DIV + Parent_share)$

Identity Check

Every transaction must satisfy:

$$GP = COS_reserve + DIV_reserve + Parent_share + Local_share$$

If this is not true, the system must reject the transaction.

No manual judgement, no negotiation, no interpretation.

8.3 Dual Reserves

Two reserves are ring-fenced and protected:



A Community Shared Workspace
Where great ideas grow!

COS Reserve – Service Guarantee

Must satisfy:

$\text{\$COS_reserve_balance} \geq \text{COS} \times \text{Credits_sold_not_yet_booked}$

Dividend Reserve – Investor Guarantee

Must satisfy:

$\text{\$DIV_reserve_balance} \geq \text{DIV} \times \text{Credits_sold_to_date}$

These reserves cannot be used for operating expenses.

8.4 Parent Revenue Policy (Simple and Universal)

A single policy applies across the entire network:

- A fixed percentage of gross revenue (default 10%) goes to the Parent CIC.

This supports:

- system maintenance
- digital infrastructure
- training
- governance
- development

The percentage is stored centrally and can only be changed via parent CIC executive decision, not per service plugin.

8.5 Local Revenue as Buffer

After allocation to COS, dividend, and parent:

- the remaining portion goes to local CIC revenue

Local revenue is the only category that may be used to fix shortfalls in protected reserves.

8.6 Reserve Priority Hierarchy

To preserve solvency:

1. COS reserve
2. Dividend reserve
3. Parent revenue
4. Local revenue

Only local revenue may be transferred when a reserve is low.

Protected reserves cannot be drawn down.

This ensures:

- guaranteed service delivery
- guaranteed investor return
- guaranteed parent funding

Before any local spending.



A Community Shared Workspace
Where great ideas grow!

8.7 Clearing & Reconciliation (Simple Rules)

Stage 1 — Point of sale

- Split happens automatically using formulas above

Stage 2 — Reserve requirement

- Reserves are compared against obligations

Stage 3 — Bank reconciliation

- Totals from ledger $\approx$ totals from bank transaction feed

If mismatch:

- flag error
 - investigation required (ISO 9001 non-conformance style)
-

8.8 Audit & Compliance Properties

This system is:

- deterministic
- transparent
- reproducible
- double-entry consistent
- hash-auditable

And aligns with:

- UK CIC reserve rules
- HMRC dividend separation
- ISO 27001 (data integrity)
- ISO 22301 (continuity and resilience)
- ISO 9001 (governance and auditability)

No special accounting judgement is required.

8.9 Neuro-Friendly Summary

When someone buys a credit:

- money comes in once
- the system automatically puts the right amounts in four pots:
 1. Service pot (can we deliver what was bought?)
 2. Investor pot (can we pay investor rewards later?)
 3. Parent pot (can we fund the network?)
 4. Local pot (money we can spend)

If any important pot is empty,
the system uses the local pot to fill it.

Protected pots stay protected.

8.10 Summary

This model makes every credit sale:

- a guaranteed service promise
- a funded investor promise



A Community Shared Workspace
Where great ideas grow!

- a stable network contribution
- a controlled local surplus

All enforced by simple, visible formulas:

- no negotiation
- no interpretation
- no discretionary risk

This is a transparent, mathematically bounded approach to community finance that is highly suitable for CIC governance, distributed systems, and external audit.

Ready to paste

You can copy this whole section into Google Docs and the equations between \$\$...\$\$ should render correctly using Auto-LaTeX Equations.

If you want, I can next:

- add a visual flow diagram (PNG or SVG) for this section
- convert to a downloadable Google Doc (.docx with rendered equations)
- update the simulation sheet to show all four pots reconciled live

Just say the word.



A Community Shared Workspace
Where great ideas grow!

Variable Classification & Control Types

How System Variables Are Grouped, Understood, and Governed

The Community Credit Exchange System uses many parameters to ensure credits remain:

- capacity-backed,
- reserve-secure,
- non-speculative, and
- easy to manage at both local and parent CIC levels.

To make this simple to operate, transparent to audit, and accessible to neurodivergent users, all variables are grouped by what they control, not by technical category alone.

This section explains the six classes of control variables, using intuitive “dashboard metaphors” and examples from the system.

9.1 Category Breakdown Overview

All variables fall into one of six groups:

1. Capacity Controls
2. Issuance Controls
3. Price / Scaling Controls
4. Reserve Controls
5. Revenue & Split Controls
6. Stimulus & Behaviour Controls

Each group controls a different aspect of system behaviour, and different automated agents interact with each group.

9.2 Capacity Controls

“Volume Adjusters” – How much can we deliver?

These variables define how much service capacity exists, either in time units, storage units, or other measurable quantities.

Typical examples:

- K_d — daily capacity
- K_window — 7-day capacity
- storage_total_gb — total gigabytes available
- concurrent_capacity — number of simultaneous bookings
- hours_open_per_day

Human metaphor:

A tap that controls how wide the service “pipe” is.

Agent responsibility:

Infrastructure agents, operations agents.



A Community Shared Workspace
Where great ideas grow!

9.3 Issuance Controls

“Pressure Adjusters” – How fast can we mint credits?

These variables control credit creation. They prevent over-issuance, hoarding, and blocked booking.

Typical examples:

- IR — issuance ratio
- α — scaling constant
- MaxCreditFraction
- C_mintable_now
- C_claim_headroom

Core identity used at issuance:

\$\$

$$C_{\text{mintable_now}} = \min(IR \times K_{\text{window}}; \text{MaxCreditFraction} \times K_{\text{window}}; R / \text{COS})$$

\$\$

Human metaphor:

A pressure gauge that regulates how quickly credits enter circulation.

Agent responsibility:

Issuance agent, stimulus agent.

9.4 Price & Scaling Controls

“Value Sliders” – What is one credit worth right now?

These variables relate to monetary scaling and value anchoring, such as inflation adjustments and currency decisions.

Typical examples:

- base_rate_per_credit (normally 1 credit = £1)
- inflation_factor
- currency_conversion_factor

Human metaphor:

A slider that adjusts value slowly, usually annually.

Agent responsibility:

Governance and policy agent only.

(These are never changed by automated logic.)

9.5 Reserve Controls

“Safety Tanks” – Where do we store the obligations?”

These protect solvency and guarantee service delivery and dividend payments.

Typical examples:

- COS_per_unit
- DIV_per_unit
- COS_reserve_balance



A Community Shared Workspace
Where great ideas grow!

- DIV_reserve_balance
- R / COS (reserve maximum on issuance)
- reserve_min_buffer

Key reserve rule:

\$\$

$$\text{COS_reserve_balance} \geq \text{COS} \times \text{sold but unbooked credits}$$

\$\$

Human metaphor:

A storage tank that ensures we can meet promised deliveries.

Agent responsibility:

Reserve agent, audit agent.

9.6 Revenue & Split Controls

“Sharing Knives” – Who gets what when money arrives?”

These variables define how gross revenue is divided at the moment of a sale.

Most important system-wide rule:

- PR — parent share rate (default 10%)

Revenue allocation equation:

\$\$

$$\text{GP} = \text{COS} + \text{DIV} + (\text{GP} \times \text{PR}) + \text{Local}$$

\$\$

Where:

- GP = gross price per unit
- COS = cost of service
- DIV = dividend allocation
- PR = parent share rate
- Local = operating surplus for local CIC

Human metaphor:

A knife that cuts one payment into many pieces.

Agent responsibility:

Payment agent, clearing agent.

9.7 Stimulus & Behaviour Controls

“Boost Buttons” – What happens when flow slows down?”

These variables respond to low utilisation, hoarding, or slow circulation.

Typical examples:

- stimulus_multiplier
- stimulus_campaign_active
- average_credit_age_days
- velocity_last_30_days
- hoarding_state



A Community Shared Workspace
Where great ideas grow!

Neuro-friendly example:

"If credits are sitting around unused,
increase issuance speed for a while,
but block share sales until utilisation improves."

Human metaphor:

A booster switch that activates when performance is low.

Agent responsibility:

Stimulus agent, utilisation agent, LLM advisor agent.

9.8 Human-Friendly Dashboard Model

Each variable class becomes a visual group on the admin dashboard:

- Blue: Capacity (volume)
- Red: Issuance (pressure)
- Green: Reserve (safety)
- Yellow: Revenue split (sharing)
- Purple: Price & scaling (value)
- Orange: Stimulus (boosters)

This allows operators to understand system state at a glance:

"Pressure is high, safety tanks are full,
no booster required."

9.9 Governance Summary

- Only governance and executive roles can modify value sliders (pricing and inflation).
- Automated agents may modify pressure adjusters and boosters within safe limits.
- No agent may modify safety tanks or sharing knives outside pre-approved policy.
- Dashboards must present all six categories separately for clarity.

These rules create:

- non-arbitrary control
 - transparency
 - safe automation
 - enforceable governance
-

9.10 Neuro-Friendly Explanation

Think of the system as water pipes.

Some knobs make the pipes bigger (capacity).

Some knobs make the water flow faster (issuance).

Some knobs change what the water is worth (pricing).

Some tanks store water safely in case we need it (reserves).

Some tools split water between people (revenue).

And one button gives extra flow when slow (stimulus).



A Community Shared Workspace
Where great ideas grow!

Every part has one job and one kind of knob.

This makes it simple to understand, simple to test, and simple to explain.

If you'd like, I can also:

- Produce a diagram visual for Section 9 (PNG or SVG)
- Turn each category into a colored dashboard tile design
- Insert specific Dgraph predicate suggestions per category
- Create a glossary page suitable for neurodivergent readers

Just say:

"Add Section 9 diagram."



A Community Shared Workspace
Where great ideas grow!

11. Distributed Storage & Triple Replication

Data storage must meet ISO/IEC 27001 continuity requirements.

Fundamental rule:

Every blob MUST have 3 replicas stored on 3 different CIC nodes.

11.1 Replica Allocation (SeaweedFS Logic)

- Replica 1 = Local CIC (if capacity exists)
- Replica 2 = CIC with greatest free storage
- Replica 3 = CIC with second-greatest free storage

If local CIC has no free space, all three replicas go to the top 3 nodes with available capacity.
Never store two replicas on same CIC node.

11.2 Storage Revenue Split

Each CIC that stores a replica earns one-third of revenue:

$$Rev(CIC_i) = \frac{Rev(blob)}{3}$$

This incentivises:

- hardware investment
 - resilience
 - distributed storage growth
-

11.3 Dynamic Capacity Throttle

If free storage drops below 5% threshold:

- Node is marked: "NO NEW REPLICAS"
- Future replica placement bypasses this node
- Credit issuance automatically throttles

This creates automatic equilibrium without manual governance.

11.4 Network Liquidity

If one CIC runs out of capacity:



A Community Shared Workspace
Where great ideas grow!

- another CIC earns the revenue
- another CIC earns the credits
- no service interruption occurs

Capacity can effectively be purchased indirectly through replica placement.
This is emergent network capacity trading.





A Community Shared Workspace
Where great ideas grow!

Dividend Reserve Mechanics

(How dividend reserves accumulate, release, and remain legally compliant under CIC rules)

Dividend reserves are the second protected financial category in the Community Credit Exchange System.

They operate similarly to Cost-of-Service (COS) reserves, but represent capital return obligations rather than service delivery obligations.

This section defines how dividend reserves are:

- created,
- tracked,
- released,
- prevented from indefinite growth,
- and protected legally under UK CIC rules.

12.1 Purpose

Dividend reserves ensure that preferential share holders receive their contractual return without jeopardising:

- operational budgets,
- cost-of-service delivery,
- or system stability.

Dividends accumulate during credit sales, and are released during formal dividend events approved by the CIC.

Dividend reserve mechanics prevent:

- underfunded dividend payouts,
- arbitrary distribution,
- or dividend promises backed by insufficient cash.

12.2 Two-Event Accounting Model

Like COS, dividend obligations arise and are satisfied in two distinct events:

A. Sale Event

Credit purchased → DIV reserve increases

B. Dividend Declaration Event

Dividend approved → DIV reserve decreases

This preserves proper separation between:

- accumulation (day-to-day credit sales), and
 - release (annual or semi-annual dividend policy decisions).
-



A Community Shared Workspace
Where great ideas grow!

12.3 Core Dividend Reserve Rule

For each credit sold:

\$\$

$\text{DIV_reserve} := \text{DIV_reserve} + \text{DIV}$

\$\$

Where:

- DIV = dividend allocation per credit, set by annual policy

Upon dividend declaration:

\$\$

$\text{DIV_reserve} := \text{DIV_reserve} - \text{Paid_DIV}$

\$\$

And:

\$\$

$\text{DIV_spent} := \text{DIV_spent} + \text{Paid_DIV}$

\$\$

This keeps dividend reserve tracking closed-loop and auditable.

12.4 Outstanding Dividend Liability

As with COS, the system tracks only the dividend yet to be released, not historical accumulation.

Outstanding dividend:

\$\$

$\text{Outstanding_DIV} = \text{DIV} \times \text{Credits_sold} - \text{DIV_spent}$

\$\$

A simple deterministic identity:

- sold credit obligation minus
- actual paid dividend

This field drives solvency signals for investor payments.

12.5 Ledger Separation

Dividend accounting requires three independent ledgers:

1. DIV Reserve Ledger
accumulates dividend allocation at sale
2. DIV Spend Ledger
reduces reserve at dividend payment event
3. Outstanding DIV Liability Ledger
tracks remaining unpaid dividend obligations

These must remain separate from COS ledgers, avoiding co-mingling of liability types.

12.6 Dividend Declaration Event

Unlike COS fulfilment (triggered by usage), dividend release is governance-triggered.



A Community Shared Workspace
Where great ideas grow!

A Dividend Declaration Event occurs:

- annually, OR
- semi-annually, OR
- per share class rules

Dividend release:

- converts protected reserve funds into spendable capital distribution
- for preferential shareholders

No interpretation is required because:

- DIV allocation per credit is deterministic
- Outstanding liability is known precisely

12.7 Dividend Policy Enforcement

Dividend payout must satisfy all three constraints:

1. DIV reserve balance
Must be $\geq$ outstanding dividend liability
2. COS priority rule
Dividend release must never reduce COS reserve
3. Parent revenue separation
Parent share cannot be used for dividend payout

If any constraint fails:

- the payment is automatically flagged and blocked
- governance must wait until next clearing cycle

No discretionary override is allowed in software.

12.8 Dividend Sustainability Indicator

Daily dashboard metric:

\$\$

$$DSI = \frac{DIV_reserve}{Outstanding_DIV}$$

\$\$

Interpretation:

- $DSI \geq 1.0 \rightarrow$ Safe to release dividends
- $0.8 \leq DSI < 1.0 \rightarrow$ Monitor and restrict issuance
- $DSI < 0.8 \rightarrow$ Block dividend payment

Neuro-friendly interpretation:

“Do we have enough saved up to pay what we owe?”

12.9 Grant & Stimulus Interaction

When grants are added to capital reserves, they may explicitly boost dividend capability:

- multiplying unit DIV allocation
- or releasing special bonus credits

BUT:

grants never alter historical obligations — only future issuance capacity.

This protects financial fairness between early and late investors.



A Community Shared Workspace
Where great ideas grow!

12.10 Example – Year-End Dividend Release

Parameters:

- Credits sold this year: 12,000
- DIV allocation per credit: £0.50
- DIV reserve balance: £8,000
- DIV spent YTD: £3,000

Outstanding liability:

\$\$

$\text{Outstanding_DIV} = (12,000 \times 0.50) - 3,000 = 3,000$

\$\$

Since DIV reserve (8,000) $\geq$ Outstanding DIV (3,000):

- Dividend declaration is permitted

The system releases:

- £3,000 to shareholders
- DIV reserve reduces to £5,000
- DIV spent increases to £6,000

Clear, deterministic, auditable.

12.11 Legal Compliance and CIC Regulation

Dividend reserves must comply with:

- CIC Asset Lock
- CIC Regulator annual report requirements
- HMRC dividend taxation rules
- IFRS revenue matching principle

Key legal statement:

Dividend reserve funds are not operating income and cannot be used for expenditure, payroll, or CIC running costs.

They are ring-fenced capital funds.

12.12 Neuro-Friendly Summary

When someone buys a credit,
we save a little bit for paying investors later.

When we agree to pay investors,
we use only that saved money.

This keeps investor payments safe, predictable, and fair.



A Community Shared Workspace
Where great ideas grow!

12.13 Implementation TODOs

- New agent: DividendAgent
 - Add fields: DIV_spent, Outstanding_DIV
 - Add dashboard indicator: DSI (Dividend Sustainability Indicator)
 - Add annual governance workflow: "Declare Dividend"
 - Add ledger sub-category for protected dividend accounts
-

12.14 Compatibility with COS Consumption Rules

Dividends must never consume COS reserves.

Software enforcement:

- COS ledger isolation
- DIV ledger isolation
- parent ledger isolation

Dividend mechanics mirror COS mechanics but are:

- time-triggered rather than usage-triggered
-

This completes the Dividend Reserve Mechanics section for the white paper.

Next recommended extension:

- Section 13: Parent Revenue Mechanics
(how parent shares are accumulated and deployed safely)

If you want this, say:

"Write Parent Revenue Mechanics."



A Community Shared Workspace
Where great ideas grow!

Parent Revenue Mechanics

(System governance, asset lock protection, and share class control with founder reward and redemption rules)

Parent revenue mechanics define how the Parent CIC receives income from local CIC trading activity, how that income is ring-fenced, and how founder and class shares are governed to maintain equity, transparency, liquidity, and CIC regulatory compliance.

13.1 Purpose

The Parent CIC exists to:

- maintain the digital infrastructure,
- enforce governance and issuance policy,
- manage shared agent libraries,
- protect reserves and audit standards,
- distribute network maintenance, bug fixing, and shared system costs.

Parent revenue must remain separate from:

- local operating revenue,
- COS (Cost-of-Service) reserves,
- dividend reserves.

Co-mingling is prohibited.

13.2 Universal Parent Revenue Rule

Every credit sale automatically allocates a fixed percentage of gross price to the Parent CIC.

Governance constant:

PR (Parent Revenue Rate) — default 10%

Recorded at point of sale:

$\text{Parent_share} = \text{GP} \times \text{PR}$

Where:

- GP = gross price per credit
- PR = parent rate (e.g. 0.10 = 10%)

This is a non-negotiable network rule, ensuring predictable income for system governance.

13.3 Ledger Categories

Parent revenue must be tracked in dedicated ledgers separate from all other reserve types:

- Parent Revenue Ledger (PRL)
- Parent Asset-Lock Ledger (PAL) — ring-fenced system ownership protection
- Redemption Reserve Ledger (RRL) — for buying back founder/community shares
- Parent Development Ledger (PDL) — optional innovation budget

These must remain segregated from:

- COS Reserve Ledger
- Dividend Reserve Ledger
- Local Revenue Ledger



A Community Shared Workspace
Where great ideas grow!

No ledger co-mingling is ever permitted.

13.4 Share Architecture (Revised)

The Parent CIC uses three share classes to balance governance, community benefit, and founder reward.

Class A — Governance & Asset-Lock Shares

- No dividend rights
- No capital gain
- One vote per share
- Ensure control and safeguarding of CIC objectives

Class F — Founder & Innovator Participation Shares

- Eligible for dividends (capped)
- Can be sold only to Parent CIC at face value (£1)
- First priority for redemption from RRL
- Vesting and lock-in periods enforce contribution commitment

Class C — Community Participation Shares

- Eligible for dividends (after Class F)
- Can request redemption (second priority)
- Represent broad community interest

Key design:

Founders are rewarded fairly,

without creating private extraction or speculative value leakage.

13.5 Dividend Pool Allocation & Cap

To maintain asset lock and prevent profit extraction:

- Only 40% of annual Parent CIC surplus may be allocated to dividends.

Deterministic definition:

$\text{Dividend_Cap} = 0.40 \times \text{Parent_Surplus}$

Where Parent Surplus = PRL balance after priority system expenses and required carry-forward.

Dividend split inside the capped pool:

- 25% → Class F (Founder & Innovator)
- 75% → Class C (Community)

Class A receives no dividends.

Software enforcement:

- If proposed distribution exceeds 40% cap → block



A Community Shared Workspace
Where great ideas grow!

- Dashboard warning → “Dividend cap exceeded”
 - Governance vote required to modify payout timing
-

13.6 Reinvestment Rule (Safety & System Growth)

Minimum 60% of annual surplus must remain in system for reinvestment.

Acceptable reinvestment uses:

- buy-back of participation shares
- distributed storage hardware
- security audit upgrades
- training and skill development
- software development
- local CIC credit acquisition

This ensures capacity growth and network sustainability.

13.7 Redemption Reserve (Liquidity Envelope)

A Redemption Reserve Ledger (RRL) provides a dedicated pool for buying back Class F and Class C shares at face value.

Redemption conditions:

- Price fixed at £1.00 per share
- Parent CIC is sole buyer
- No external transfers

Redemption priority order:

1. Class F (Founder & Innovator)
2. Class C (Community)

Redemption must stop automatically if:

- COS liability is underfunded
- Dividend liability is underfunded
- Continuity reserve is breached
- RRL balance exceeds safety threshold

Redemption requests can be queued.

13.8 Bank Handling (Standing Order Model)

Parent revenue flows from local CIC bank accounts via:

- Daily BACS batches
- Standing orders
- No instant card payment fees

This produces:

- Lower transaction cost
 - Cleaner audit trails
 - Daily reconciliation checks
-



A Community Shared Workspace

Where great ideas grow!

13.9 Deterministic Allocation of Parent Revenue

At point of credit sale:

Parent_amount = GP × PR

Automatically posted to:

- PRL
- PAL entry
- RRL % update (as configured)

No human override permitted.

Exported metadata includes:

- CIC ID
- Plugin ID
- Bank reference
- Timestamp
- Member wallet ID

All of this is hash-linked for audit.

13.10 Parent CIC Cashflow Example

Local annual credit sales: £250,000

PR = 10%

Parent annual revenue = £25,000

Full sequence allocation:

Priority	Purpose	Amount
1	Infrastructure & hosting	£10,000
2	Security / ISO audits	£3,000
3	Governance / annual return	£2,000
4	Continuity reserve carry-forward	£5,000
5	Agent library development	£2,000
6	Dividend cap (40% of surplus)	£1,200
7	Redemption reserve top-up (optional)	balance

13.11 Non-Speculative Share Transfer Rules

Founder & Community participation shares:

- Cannot be sold on open market
- Cannot be priced above £1
- Must be offered to Parent CIC first
- May only be transferred externally if redemption reserve is empty



A Community Shared Workspace
Where great ideas grow!

This prevents special privilege leakage.

13.12 Neuro-Friendly Summary

Every time someone buys credits,
a small part goes to the main organisation.
This keeps the system running.
Some of that money can be shared out,
but only a limited amount each year.

You can sell back your shares,
but only for £1 each
and only to the system —
not to private buyers.

This keeps everything fair
and stops anyone taking control for money.

13.13 Implementation TODOs

- Add schema fields:
 - PR_rate
 - PR_reserve_ledger
 - RRL_balance
 - redemption_queue
 - share_class
 - vesting_period
 - Add agents:
 - ParentAccountingAgent
 - ShareRedemptionAgent
 - Add system triggers:
 - “Dividend cap exceeded”
 - “Continuity reserve alert”
 - “Redemption paused — reserve under threshold”
 - Add Dgraph types:
 - ShareLedger
 - RedemptionRequest
 - FounderShareBlock
-

13.14 Cross-Link With Sections 11 & 12

- Parent revenue may never be used to top up COS or DIV reserves.
- COS and DIV remain independent safety systems.
- Parent revenue is applied only after operational safety and reserve priority conditions are met.

This establishes deterministic multi-ledger integrity across:



A Community Shared Workspace
Where great ideas grow!

- COS Reserve (service safety)
- DIV Reserve (investor / share safety)
- PR Ledger (system safety)
- RRL Ledger (share liquidity)

This completes the updated Section 13 — Parent Revenue Mechanics
with founder reward + redemption reserve + share priority logic fully integrated.

If you want next, I can write:

- Founder Share Vesting Policy (2 years lock-in, quarterly vesting)
- Family Trust Integration Annex
- Constitution wording for Articles of Association
- “Redemption Reserve Specification” as a separate section
-



A Community Shared Workspace
Where great ideas grow!

12. Personal vs Business Mode (Tax Compliance)

Every transaction must require the user to choose:

- Personal Mode
- Business Mode

12.1 Personal Mode

Used when:

- Non-commercial activity
- Neighbourly exchange
- Community access without trade intention

HMRC position:

Typically non-taxable (similar to LETS and informal non-business barter).

12.2 Business Mode

Used when:

- The CIC delivers a taxable service
- A trader uses credits as part of their business
- Paid training, wellbeing, services, consulting, retail, etc.

HMRC treatment:

Credits are non-cash consideration (barter).

The GBP equivalent value must be recorded.

The Credit Exchange MUST assign a valuation in £ when business mode is selected.

12.3 Accounting Integration

When business mode is selected:

- Post transaction to accounting system (Xero / QuickBooks / Sage)
- Include:
 - Income account
 - Expense account
 - VAT code
 - GBP valuation

This integrates seamlessly with Making Tax Digital (MTD) protocols.

No equation block required here.



A Community Shared Workspace
Where great ideas grow!

13. Member Wallet Controls

Each member wallet holds:

- Credits balance
- Transaction history
- Mode per transaction (Personal/Business)
- Pending (provisional) credits

Wallet also manages:

- Transfers
- Spending
- Selling (GBP redemption)

No math here — pure interface logic.



A Community Shared Workspace
Where great ideas grow!

14. Trading vs Selling Credits

There are two distinct actions:

1. Trade credits
 - member ↔ member
 - or member ↔ local CIC plugin
 - no GBP movement needed
 - increases community velocity
2. Sell credits
 - member → CIC / CIC → member
 - GBP movement occurs
 - reserves tracked
 - taxable if business mode is selected

Reserve rule on sale:

$$ReserveBalance_{after\ sale} \geq RequiredReserves$$

If not, no sale permitted (automatic lockout).

This protects:

- solvency
 - audit integrity
 - FCA compliance
-



A Community Shared Workspace
Where great ideas grow!

15. Community Wealth and Velocity

The purpose of the InQbeta Credit System is not to create scarcity, but to create community circulation.

Community wealth can be estimated as:

$$W = C \times V \times U \times F$$

Where:

- C = credits in circulation
- V = velocity (transactions per period)
- U = utilisation of available capacity
- F = internal face value

Interpretation:

- Increase transactions → increase wealth
- Increase utilisation → increase wealth
- Increase capacity → increase wealth

This is a macroeconomic KPI for CIC boards.



A Community Shared Workspace
Where great ideas grow!

16. ISO Audit & Hash-Linked Ledger

Every credit action yields two ledger entries (double entry accounting):

- Debit entry
- Credit entry

Hash chain audit rule:

$$H_n = Hash(T_n + H_{n-1})$$

Where:

- H_n = block hash at time n
- T_n = transaction summary at time n
- H_{n-1} = previous block hash

This gives:

- Tamper-evident ledger
- Full auditability without blockchain complexity
- Zero retroactive edit ability

This satisfies ISO/IEC 27001 data integrity controls.



A Community Shared Workspace
Where great ideas grow!

17. Governance Controls (ISO-Style)

Board approval required for:

- Changing COSReserveRatio
- Changing IssuanceRatio (IR)
- Adding/removing Service Plugins
- Allocating grant reserves
- Hardware procurement > £500

Required governance documentation:

- Risk register
- Change control log
- Board resolution record
- Annual reserve reconciliation report
- Internal audit checklist

(All format templates can be produced later.)

No equation block required.



A Community Shared Workspace
Where great ideas grow!

18. Regulatory Position (UK Summary)

Not a security

No appreciation, no dividend yield, no speculation.

Not electronic money

Credits are service vouchers, not stored GBP value.

Barter and consideration

Business transactions are treated as barter trades (HMRC).

CIC asset lock compliant

All benefits remain for community interest.

GDPR compliant

Distributed storage uses encrypted blob replication across UK-based nodes.



A Community Shared Workspace
Where great ideas grow!

19. Conclusion

The InQbeta Credit System is:

- A fair capacity-backed value system
- 100% reserve and queue-safe
- Designed for ISO audit friendliness
- Legally compatible with CIC structures
- Transparent for members, boards, and regulators

It solves historic problems in:

- LETS
- Mutual credit
- Community barter systems
- Social tokens

...by adding:

- hard issuance limits
- reserve underwriting
- dynamic capacity throttling
- triple-replica distributed storage
- banking reconciliation
- ISO control structure

Result:

A non-speculative, adaptable, and resilient local community economy,
with real-world value tied to real-world deliverable services.
